Data Schema Specification — Gambling Levy Portal

Data Schema Specification – Gambling Levy Portal

Document Version: 2.1 | **Date:** June 2026

> This document describes the data model underpinning the Gambling Levy Portal. It is the companion narrative to Gambling_Levy_Portal_ERD.pdf and Entity Relationship Diagram Table.xlsx, restating the schema as prose for developers, architects, and DHSC stakeholders reviewing the design ahead of build.

Table of Contents

- 1. Overview
  - 1.1 Purpose
  - 1.2 Product context
  - 1.3 Domain model at a glance
- 2. Domain 1 — Master Data
  - 2.1 Org
  - 2.2 User
  - 2.3 UserOrgMembership
  - 2.4 Region
  - 2.5 OrgNeighbour
- 3. Domain 2 — Collection Configuration
  - 3.1 Collection
  - 3.2 CollectionOrg
  - 3.3 CollectionReminder
  - 3.4 FormVersion
  - 3.5 Section
  - 3.6 FormSectionCategory
  - 3.7 Question
  - 3.8 QuestionDependency
  - 3.9 Component
  - 3.10 DataList

    - 3.11 DataListItem
- 4. Domain 3 — Activities and Key Results
    - 4.1 Category
    - 4.2 Task
    - 4.3 Activity
    - 4.4 ActivityPartnership
    - 4.5 ActivityKeyResult
    - 4.6 KeyResult
    - 4.7 KeyResultVersion
    - 4.8 KeyResultQuestion
    - 4.9 KeyResultCategory
    - 4.10 KeyResultTask
    - 4.11 KeyResultEligibility
    - 4.12 KeyResultDependency
    - 4.13 KeyResultReviewLog
- 5. Domain 4 — Transactional
    - 5.1 Submission
    - 5.2 SubmissionValue
    - 5.3 SpendReport
    - 5.4 SubmissionExport
- 6. Domain 5 — Longitudinal and Audit
    - 6.1 MaturityRecord
    - 6.2 MaturityAnswer
    - 6.3 EventLog
    - 6.4 Feedback
- 7. Value Objects and Enumerations
- 8. Entity Relationship Overview
- 9. Examples
    - 9.1 KR-driven quarterly return generation
    - 9.2 Partnership activity
    - 9.3 Proxy submission
    - 9.4 Annual Maturity Survey lifecycle
- 10. Key Design Decisions
- 11. Open Questions and Assumptions
- Changelog

---

1. Overview

1.1 Purpose

The schema underpins the Gambling Levy Portal — a DHSC service for managing grant reporting on gambling harms prevention across ~153 Local Authorities and participating Voluntary, Community and Social Enterprise (VCSE) organisations. It is optimised for three concerns:

- **No-code configuration** — DHSC admins can define collections, forms, Key Results, conditional logic, and reminders without code deployment.
- **Historic fidelity** — submissions render against the exact form and Key Result definitions that were live when they were completed.
- **Regulatory traceability** — field-level audit, explicit review logs, and proxy-submission attribution sufficient for levy oversight.

1.2 Product context

The portal manages three distinct collection types, each represented by a `Collection` record with a different `type`:

Collection type	Cadence	What it captures
Annual Maturity Survey	Living profile, confirmed annually	Each LA's strategic approach to gambling harms. Stored in `MaturityRecord` + `MaturityAnswer` rather than `Submission` to reflect its different lifecycle.
Activities Registry	Static, updated as activities are added or withdrawn	Each org's registered prevention projects, each linked to DHSC-defined `Category`, `Task`, and one or more `KeyResult`s.
Quarterly Reporting Collection	Quarterly	Financial and performance return dynamically shaped by each org's registered Activities and their selected Key Results.

1.3 Domain model at a glance

35 entities organised into five domains:

Domain	Entities	Characteristics
Master data	5	Stable reference records. Managed by DHSC admins or imported from external tools.
Collection configuration	11	No-code definition of reporting structure. Versioned to protect historic submissions.
Activities & Key Results	13	Bridge between configuration and transactional data. The mechanism by which quarterly forms are dynamically generated.
Transactional	4	Submitted data. Immutable once submitted.
Longitudinal & audit	4	Long-term tracking and governance records. Append-only or slowly updated.

Cross-cutting conventions:

- **Auditable** — most entities carry `created_at` / `updated_at`, and where relevant `created_by` referencing `User`.
- **Soft deletion** — inactive records retained via `is_active` (and `archived_at` / `archived_by` on `KeyResult`). Hard deletes would break rendering of historic submissions.
- **Versioning** — `FormVersion` and `KeyResultVersion` capture point-in-time definitions so submissions and activity registrations never silently change when admins edit definitions.
- **Polymorphic audit** — `EventLog` captures field-level diffs across every entity via `entity_type` + `entity_id`.

---

2. Domain 1 — Master Data

Core reference entities that underpin the whole system. Stable, slowly changing records managed primarily by DHSC administrators.

2.1 Org

Represents a council, charity, or reporting body. The root entity for all data ownership, reporting attribution, and funding allocation.

Property	Type	Description
`org_id`	`uuid` PK	Unique identifier
`org_name`	`string`	Display name
`org_type`	`OrgType` enum	`LA` or `VCSE`
`region_id`	`uuid` FK → Region	Geographic region
`ons_code`	`string?`	ONS geographic identifier
`vendor_id`	`string?`	DHSC internal finance identifier, used in spend exports
`total_fund`	`decimal?`	Allocated funding (GBP). A single scalar — multi-year allocation history is not modelled in v1
`onboarding_status`	`OnboardingStatus` enum	`invited` | `active` | `suspended`
`created_at` / `updated_at`	`datetime`	Audit fields

Relationships: 1:N `UserOrgMembership`, `Activity`, `Submission`, `MaturityRecord`, `Feedback`, `OrgNeighbour` (as org or neighbour); N:1 `Region`.

2.2 User

Individual user accessing the portal.

Property	Type	Description
`user_id`	`uuid` PK	Unique identifier
`full_name`	`string`	
`email`	`string`	Federated identity — no password stored locally
`mfa_status`	`MfaStatus` enum	Tracks custom TOTP MFA enrolment (see MFA-Implementation.md)
`contact_number`	`string?`	From account-setup epic
`job_title`	`string?`	From account-setup epic
`is_active`	`bool`	Soft-deactivation flag
`last_login_at`	`datetime?`	
`created_at`	`datetime`	

Relationships: 1:N `UserOrgMembership`, `Submission` (as submitter), `EventLog`, `Feedback`, and every `created_by` / `reviewed_by` / `imported_by` FK across the schema.

2.3 UserOrgMembership

Join table linking users to one or more organisations. Supports proxy submission and role-based access.

Property	Type	Description
`membership_id`	`uuid` PK	
`user_id`	`uuid` FK → User	
`org_id`	`uuid` FK → Org	
`role`	`Role` enum	`lead` | `contributor` | `finance` | `read_only`
`is_active`	`bool`	
`created_at` / `updated_at`	`datetime`	

A user may hold memberships in multiple organisations simultaneously. Role is per-membership, not per-user.

2.4 Region

Geographic region classification of organisations.

Property	Type	Description
`region_id`	`uuid` PK	
`name`	`string`	e.g. "North West", "London"
`region_type`	`RegionType` enum	`geographic` | `statistical`
`created_at`	`datetime`	

Region groups LAs for the geographic-neighbour filter on the maturity benchmarking dashboard. Statistical-neighbour relationships are handled via `OrgNeighbour`, not via region.

2.5 OrgNeighbour

Org-to-org neighbour mapping. Stores both geographic and statistical neighbour pairs.

Property	Type	Description
`id`	`uuid` PK	
`org_id`	`uuid` FK → Org	
`neighbour_org_id`	`uuid` FK → Org	
`source`	`NeighbourSource` enum	`statistical` | `geographic`
`imported_at`	`datetime`	Per-import audit stamp
`imported_by`	`uuid` FK → User	Admin who ran the import

Statistical-neighbour pairs are loaded by DHSC admins from an externally produced reference dataset rather than derived internally from `Org` attributes — see §10, Key Design Decisions.

---

3. Domain 2 — Collection Configuration

Entities that allow DHSC administrators to define, version, and manage reporting windows and the forms within them — without developer intervention.

3.1 Collection

A reporting window. One Collection represents one maturity survey, activities-registry window, or quarterly reporting cycle.

Property	Type	Description
`collection_id`	`uuid` PK	
`name`	`string`	
`type`	`CollectionType` enum	`quarterly` | `maturity` | `activities`
`status`	`CollectionStatus` enum	`draft` | `scheduled` | `open` | `closed` | `archived`
`start_date` / `end_date`	`date`	
`financial_year`	`string`	e.g. "2026/27"
`description`	`string?`	
`instructions_html`	`string?`	Admin-authored guidance shown to submitters
`is_dhsc_managed`	`bool`	
`created_by`	`uuid` FK → User	
`manually_closed_at`	`datetime?`	Emergency override — admin closed before `end_date`
`manually_closed_by`	`uuid?` FK → User	
`extended_to`	`date?`	Emergency override — admin extended `end_date`
`extended_by`	`uuid?` FK → User	
`created_at` / `published_at`	`datetime`	

3.2 CollectionOrg

Scopes a collection to specific organisations and tracks per-org submission status.

Property	Type	Description
`collection_org_id`	`uuid` PK	
`collection_id`	`uuid` FK → Collection	
`org_id`	`uuid` FK → Org	
`is_required`	`bool`	
`due_date_override`	`date?`	Per-org deadline overriding `Collection.end_date`
`reminder_sent_at`	`datetime?`	
`status`	`CollectionOrgStatus` enum	`not_started` | `in_progress` | `submitted` | `accepted` | `overdue`
`created_at` / `updated_at`	`datetime`	

3.3 CollectionReminder

Configurable automated reminder rules. Enables DHSC admins to schedule nudges without developer involvement.

Property	Type	Description
`reminder_id`	`uuid` PK	
`collection_id`	`uuid` FK → Collection	
`trigger_type`	`ReminderTrigger` enum	`days_before_close` | `days_after_open` | `manual`
`trigger_value`	`int?`	Number of days (null when `manual`)
`message_template`	`string`	
`sent_at`	`datetime?`	Populated when the reminder has fired
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

3.4 FormVersion

Versioned form definition. Ensures historic submissions are always rendered against the form schema that was live when they were completed.

Property	Type	Description
`form_version_id`	`uuid` PK	
`collection_id`	`uuid` FK → Collection	
`version_number`	`int`	
`version_label`	`string?`	
`change_summary`	`string?`	
`published_at`	`datetime?`	
`is_active`	`bool`	
`created_by`	`uuid` FK → User	
`superseded_by`	`uuid?` FK → FormVersion	Chain pointer to the version that replaced this one
`created_at` / `updated_at`	`datetime`	

Submissions are locked to the `form_version_id` that was live at submission time (see `Submission.form_version_id`).

3.5 Section

A page or grouped block within a form.

Property	Type	Description
`section_id`	`uuid` PK	
`form_version_id`	`uuid` FK → FormVersion	
`title`	`string`	
`description`	`string?`	
`sequence_order`	`int`	
`display_logic`	`json?`	Section-level conditional visibility
`is_repeatable`	`bool`	If true, the section renders once per row in an associated list (e.g. per registered activity)
`repeat_label`	`string?`	
`repeat_min` / `repeat_max`	`int?`	
`is_optional`	`bool`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	

3.6 FormSectionCategory

Links sections to DHSC policy categories so aggregated outputs can be sliced by theme in PowerBI exports.

Property	Type	Description
`id`	`uuid` PK	
`section_id`	`uuid` FK → Section	
`category_id`	`uuid` FK → Category	
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

3.7 Question

An individual question. A question defines the data being requested and any guard logic; the input control lives on `Component`.

Property	Type	Description
`question_id`	`uuid` PK	
`section_id`	`uuid` FK → Section	
`task_id`	`uuid?` FK → Task	Links a question to a policy task for downstream analysis
`question_text`	`string`	
`hint_text`	`string?`	
`guidance_html`	`string?`	Rich-text guidance shown alongside the field
`guard_logic`	`json?`	Fallback JSON for compound logic the simple dependency model can't express
`display_logic`	`json?`	Question-level conditional visibility
`is_optional`	`bool`	
`is_repeatable`	`bool`	
`sequence_order`	`int`	
`tag`	`QuestionTag` enum	`spend` | `output` | `narrative` | `indicator` — drives peer-view redaction
`created_by`	`uuid` FK → User	
`created_at` / `updated_at` / `archived_at`	`datetime`	

Narrative-tagged answers are redacted when Activities are viewed in the peer directory.

3.8 QuestionDependency

Explicit row-per-rule conditional logic. Separates branching rules from question definitions so admins can read, edit, and delete conditions without touching JSON.

Property	Type	Description
`dependency_id`	`uuid` PK	
`parent_question_id`	`uuid` FK → Question	Question whose answer is evaluated
`dependent_question_id`	`uuid` FK → Question	Question whose visibility / requirement changes
`condition_operator`	`ConditionOperator` enum	`equals` | `not_equals` | `contains` | `greater_than` | `less_than` | `is_answered` | `is_empty`
`condition_value`	`string?`	Compared against the parent answer
`action`	`DependencyAction` enum	`show` | `hide` | `require` | `disable` | `nest`
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

Complex compound logic falls back to `Question.guard_logic`.

3.9 Component

The input control for a question. Holds all validation and rendering behaviour.

Property	Type	Description
`component_id`	`uuid` PK	
`question_id`	`uuid` FK → Question	
`type`	`ComponentType` enum	`text` | `textarea` | `number` | `currency` | `currency_masked` | `date` | `date_range` | `radio` | `checkbox` | `select` | `multi_select` | `file_upload` | `url` | `percentage`
`validation_rules`	`json?`	Residual JSON for rules not expressible as scalars below
`list_id`	`uuid?` FK → DataList	Required for radio / select / checkbox / multi_select
`placeholder_text`	`string?`	
`prefix_label` / `suffix_label`	`string?`	e.g. "£", "%"
`min_value` / `max_value`	`decimal?`	
`min_length` / `max_length`	`int?`	
`min_date` / `max_date`	`date?`	
`is_required`	`bool`	
`sequence_order`	`int`	
`created_at` / `updated_at`	`datetime`	

`currency_masked` is distinct from `currency` to reflect different input behaviour for spend fields (e.g. masked entry, rounded display).

3.10 DataList

A reusable option set for radio, select, checkbox, and multi-select components. Managed centrally by DHSC.

Property	Type	Description
`list_id`	`uuid` PK	
`list_name`	`string`	
`description`	`string?`	
`is_dhsc_managed`	`bool`	
`is_active`	`bool`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	

3.11 DataListItem

Individual selectable option within a DataList.

Property	Type	Description
`item_id`	`uuid` PK	
`list_id`	`uuid` FK → DataList	
`label`	`string`	Display label
`value`	`string`	Stored value when selected
`hint_text`	`string?`	
`sequence_order`	`int`	
`is_active`	`bool`	Soft-delete flag — retired items remain in the DB so historic submissions retain their original labels
`created_at` / `updated_at`	`datetime`	

---

4. Domain 3 — Activities and Key Results

Entities that manage the static activity registry and the DHSC-defined Key Result framework. The Activity-to-KeyResult relationship is the mechanism by which quarterly reporting forms are dynamically generated.

4.1 Category

High-level policy theme (e.g. Education, Treatment).

Property	Type	Description
`category_id`	`uuid` PK	
`name`	`string`	
`description`	`string?`	
`is_active`	`bool`	
`sequence_order`	`int`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	

4.2 Task

Sub-area within a category.

Property	Type	Description
`task_id`	`uuid` PK	
`category_id`	`uuid` FK → Category	
`task_name`	`string`	
`description`	`string?`	
`sequence_order`	`int`	
`is_active`	`bool`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	

Tasks narrow the set of Key Results surfaced to an LA when registering an activity, keeping the selection UI manageable.

4.3 Activity

A specific prevention project registered by an LA or VCSE.

Property	Type	Description
`activity_id`	`uuid` PK	
`org_id`	`uuid` FK → Org	
`title`	`string`	
`description`	`string?`	
`category_id`	`uuid` FK → Category	
`task_id`	`uuid` FK → Task	
`status`	`ActivityStatus` enum	`draft` | `active` | `completed` | `withdrawn`
`is_lead`	`bool`	True if this Org is the lead on a partnership activity
`lead_org_id`	`uuid?` FK → Org	Populated on partner-side records pointing to the lead Org
`custom_category_text`	`string?`	"Other" workflow — free text when `category_id` maps to an "Other" placeholder
`custom_task_text`	`string?`	As above for Task
`is_flagged_for_review`	`bool`	Custom categories / tasks / KRs surface this to DHSC
`is_visible_in_directory`	`bool`	Controls appearance in the peer activity directory
`created_at` / `updated_at`	`datetime`	

4.4 ActivityPartnership

Resolves the lead/partner relationship for shared regional activities.

Property	Type	Description
`id`	`uuid` PK	
`lead_activity_id`	`uuid` FK → Activity	The lead Org's Activity record
`partner_org_id`	`uuid` FK → Org	
`status`	`PartnershipStatus` enum	`pending` | `active` | `declined`
`invited_by`	`uuid` FK → User	
`created_at`	`datetime`	

Partner LAs receive a read-only synced view of the lead's activity. The lead LA owns the qualitative reporting.

4.5 ActivityKeyResult

Records which Key Results an Org has committed to against an activity. `kr_version_id` freezes the KR definition at the point of selection.

Property	Type	Description
`id`	`uuid` PK	
`activity_id`	`uuid` FK → Activity	
`kr_id`	`uuid` FK → KeyResult	
`kr_version_id`	`uuid` FK → KeyResultVersion	Snapshot of the KR at selection time
`is_custom`	`bool`	True for free-text "Other" KRs
`custom_kr_text`	`string?`	
`status`	`ActivityKRStatus` enum	`active` | `withdrawn` | `superseded`
`created_by`	`uuid` FK → User	
`created_at` / `withdrawn_at`	`datetime`	
`withdrawal_reason`	`string?`	

4.6 KeyResult

A standardised measurable result defined by DHSC.

Property	Type	Description
`kr_id`	`uuid` PK	
`label`	`string`	
`description`	`string?`	
`guidance_html`	`string?`	
`metric_type`	`MetricType` enum	`numeric` | `percentage` | `boolean` | `currency` | `text`
`unit_label`	`string?`	e.g. "referrals", "£"
`is_standard`	`bool`	Distinguishes DHSC-curated KRs from custom ones
`is_active`	`bool`	
`sequence_order`	`int`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	
`archived_at`	`datetime?`	
`archived_by`	`uuid?` FK → User	
`archive_reason`	`string?`	

4.7 KeyResultVersion

Point-in-time snapshot of a KR definition. Protects historic activity registrations when DHSC updates a KR label or metric type.

Property	Type	Description
`kr_version_id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`version_number`	`int`	
`label`	`string`	Frozen label
`description`	`string?`	
`guidance_html`	`string?`	
`metric_type`	`MetricType` enum	
`unit_label`	`string?`	
`change_summary`	`string?`	
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	
`is_active`	`bool`	

4.8 KeyResultQuestion

Explicit mapping between a KR and the questions it injects into the quarterly return. ***This is the mechanism behind dynamic form generation*** *— no code change required when DHSC adds or removes a question from a KR.*

Property	Type	Description
`kr_question_id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`question_id`	`uuid` FK → Question	
`is_required`	`bool`	
`sequence_order`	`int`	
`created_by`	`uuid` FK → User	
`created_at` / `updated_at`	`datetime`	

4.9 KeyResultCategory

Allows a KR to span multiple policy categories (many-to-many).

Property	Type	Description
`id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`category_id`	`uuid` FK → Category	
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

4.10 KeyResultTask

Scopes a KR to specific tasks. Only KRs mapped to the selected task are shown when an LA registers an activity.

Property	Type	Description
`id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`task_id`	`uuid` FK → Task	
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

4.11 KeyResultEligibility

Controls which org types or regions can select a KR. Supports DHSC piloting a KR in specific regions before national rollout.

Property	Type	Description
`id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`org_type`	`OrgTypeEligibility` enum	`LA` | `VCSE` | `all`
`region_id`	`uuid?` FK → Region	Null means all regions
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

4.12 KeyResultDependency

Requires or excludes relationships between KRs. Enforced during activity registration UI without hardcoding rules in application logic.

Property	Type	Description
`id`	`uuid` PK	
`parent_kr_id`	`uuid` FK → KeyResult	
`dependent_kr_id`	`uuid` FK → KeyResult	
`dependency_type`	`KRDependencyType` enum	`requires` | `excludes`
`created_by`	`uuid` FK → User	
`created_at`	`datetime`	

4.13 KeyResultReviewLog

Human-readable audit trail for all admin actions on a KR. Captures intent and rationale, complementing the field-level diffs in `EventLog`.

Property	Type	Description
`review_id`	`uuid` PK	
`kr_id`	`uuid` FK → KeyResult	
`reviewed_by`	`uuid` FK → User	
`review_type`	`KRReviewType` enum	`created` | `updated` | `archived` | `reinstated` | `scoping_changed`
`notes`	`string?`	
`reviewed_at`	`datetime`	

---

5. Domain 4 — Transactional

Entities that capture the actual submitted data. Designed to be immutable once submitted, with status fields tracking the digital lifecycle of each record.

5.1 Submission

A frozen submitted report against a specific `FormVersion` of a `Collection`.

Property	Type	Description
`submission_id`	`uuid` PK	
`org_id`	`uuid` FK → Org	The data-owning organisation
`collection_id`	`uuid` FK → Collection	
`form_version_id`	`uuid` FK → FormVersion	Frozen at submission time
`user_id`	`uuid` FK → User	The submitter
`proxy_org_id`	`uuid?` FK → Org	If set, the submitter acted on behalf of this Org — for audit of proxy flows
`submitted_at`	`datetime?`	
`status`	`SubmissionStatus` enum	`draft` | `submitted` | `accepted` | `returned`
`snapshot_id`	`uuid?`	Immutable snapshot reference
`created_at` / `updated_at`	`datetime`	

Proxy semantics: `org_id` always refers to the data-owning Org; `proxy_org_id` records the acting Org when different. DHSC can always see both "who submitted" (User) and "which legal entity owns the data" (Org) in a single record.

5.2 SubmissionValue

An individual answer within a submission.

Property	Type	Description
`value_id`	`uuid` PK	
`submission_id`	`uuid` FK → Submission	
`component_id`	`uuid` FK → Component	
`activity_id`	`uuid?` FK → Activity	Set only on activity-linked answers in quarterly returns
`value_type`	`string`	Mirrors the component type so the value can be interpreted without joining
`reported_value`	`string`	Stored representation
`currency`	`string?`	Populated for currency / currency_masked components
`created_at` / `updated_at`	`datetime`	

5.3 SpendReport

Records what the organisation spent in the last quarter. ***Status tracks the digital submission lifecycle only*** *— payment processing is handled entirely offline by DHSC finance using existing ERP systems.*

Property	Type	Description
`spend_report_id`	`uuid` PK	
`submission_id`	`uuid` FK → Submission	
`org_id`	`uuid` FK → Org	
`reporting_period_start` / `reporting_period_end`	`date`	
`total_reported_spend`	`decimal`	
`submitted_at`	`datetime?`	
`status`	`SpendReportStatus` enum	`draft` | `submitted` | `accepted` | `queried`

5.4 SubmissionExport

Auditable record of every download generated from the portal. Includes the yearly CFO summary document.

Property	Type	Description
`export_id`	`uuid` PK	
`submission_id`	`uuid?` FK → Submission	Null for yearly summaries that span multiple submissions
`org_id`	`uuid` FK → Org	
`export_type`	`ExportType` enum	`quarterly_pdf` | `yearly_summary_pdf` | `yearly_summary_csv` | `admin_download`
`generated_by`	`uuid` FK → User	
`generated_at`	`datetime`	
`financial_year`	`string?`	Populated on yearly summaries

The **yearly summary PDF** is the document the CFO signs offline. The portal generates it; the signed document is returned to DHSC by email and is *not* ingested back into the portal.

---

6. Domain 5 — Longitudinal and Audit

Entities that support long-term tracking, governance, and service improvement. Append-only or slowly-updated records.

6.1 MaturityRecord

A longitudinal maturity record per organisation. Stored separately from `Submission` to reflect the annual-confirmation lifecycle of the maturity survey.

Property	Type	Description
`record_id`	`uuid` PK	
`org_id`	`uuid` FK → Org	
`effective_from` / `effective_to`	`date`	Supports year-on-year maturity tracking
`status`	`MaturityStatus` enum	`draft` | `submitted` | `accepted`
`created_at` / `updated_at`	`datetime`	

6.2 MaturityAnswer

An individual answer within a maturity record.

Property	Type	Description
`answer_id`	`uuid` PK	
`record_id`	`uuid` FK → MaturityRecord	
`question_id`	`uuid` FK → Question	Same Question definitions as the transactional forms
`value`	`string`	
`updated_at`	`datetime`	

6.3 EventLog

Polymorphic field-level audit log. Critical for regulatory traceability of the levy.

Property	Type	Description
`event_id`	`uuid` PK	
`entity_type`	`string`	Logical entity name (e.g. "Activity", "KeyResult")
`entity_id`	`uuid`	PK of the affected entity
`field_name`	`string?`	Null for create/delete events
`old_value`	`string?`	
`new_value`	`string?`	
`user_id`	`uuid` FK → User	
`timestamp`	`datetime`	
`change_reason`	`string?`	

Captures field-level diffs automatically. `KeyResultReviewLog` complements this with admin-authored intent notes.

6.4 Feedback

Post-submission and in-journey service feedback.

Property	Type	Description
`feedback_id`	`uuid` PK	
`user_id`	`uuid` FK → User	
`org_id`	`uuid` FK → Org	
`submission_id`	`uuid?` FK → Submission	
`feedback_type`	`FeedbackType` enum	`in_journey` | `post_submission`
`page_context`	`string?`	Route or page identifier for hot reports
`question_id`	`uuid?` FK → Question	Links confusion to a specific field
`score`	`int?`	Satisfaction score for `post_submission`
`comments`	`string?`	
`is_reviewed`	`bool`	
`reviewed_by`	`uuid?` FK → User	
`reviewed_at`	`datetime?`	
`timestamp`	`datetime`	

---

7. Value Objects and Enumerations

Consolidated reference of all enumerated values across the schema.

7.1 Master data

Enum	Values
`OrgType`	`LA` , `VCSE`
`OnboardingStatus`	`invited` , `active` , `suspended`
`MfaStatus`	`not_enrolled` , `pending` , `enrolled` , `disabled` *(exact values reconciled in MFA-Implementation.md)*
`Role`	`lead` , `contributor` , `finance` , `read_only`
`RegionType`	`geographic` , `statistical`
`NeighbourSource`	`statistical` , `geographic`

7.2 Collection configuration

Enum	Values
`CollectionType`	`quarterly`, `maturity`, `activities`
`CollectionStatus`	`draft`, `scheduled`, `open`, `closed`, `archived`
`CollectionOrgStatus`	`not_started`, `in_progress`, `submitted`, `accepted`, `overdue`
`ReminderTrigger`	`days_before_close`, `days_after_open`, `manual`
`QuestionTag`	`spend`, `output`, `narrative`, `indicator`
`ConditionOperator`	`equals`, `not_equals`, `contains`, `greater_than`, `less_than`, `is_answered`, `is_empty`
`DependencyAction`	`show`, `hide`, `require`, `disable`, `nest`
`ComponentType`	`text`, `textarea`, `number`, `currency`, `currency_masked`, `date`, `date_range`, `radio`, `checkbox`, `select`, `multi_select`, `file_upload`, `url`, `percentage`

7.3 Activities and Key Results

Enum	Values
`ActivityStatus`	`draft`, `active`, `completed`, `withdrawn`
`PartnershipStatus`	`pending`, `active`, `declined`
`ActivityKRStatus`	`active`, `withdrawn`, `superseded`
`MetricType`	`numeric`, `percentage`, `boolean`, `currency`, `text`
`OrgTypeEligibility`	`LA`, `VCSE`, `all`
`KRDependencyType`	`requires`, `excludes`
`KRReviewType`	`created`, `updated`, `archived`, `reinstated`, `scoping_changed`

7.4 Transactional

Enum	Values
`SubmissionStatus`	`draft`, `submitted`, `accepted`, `returned`
`SpendReportStatus`	`draft`, `submitted`, `accepted`, `queried`
`ExportType`	`quarterly_pdf`, `yearly_summary_pdf`, `yearly_summary_csv`, `admin_download`

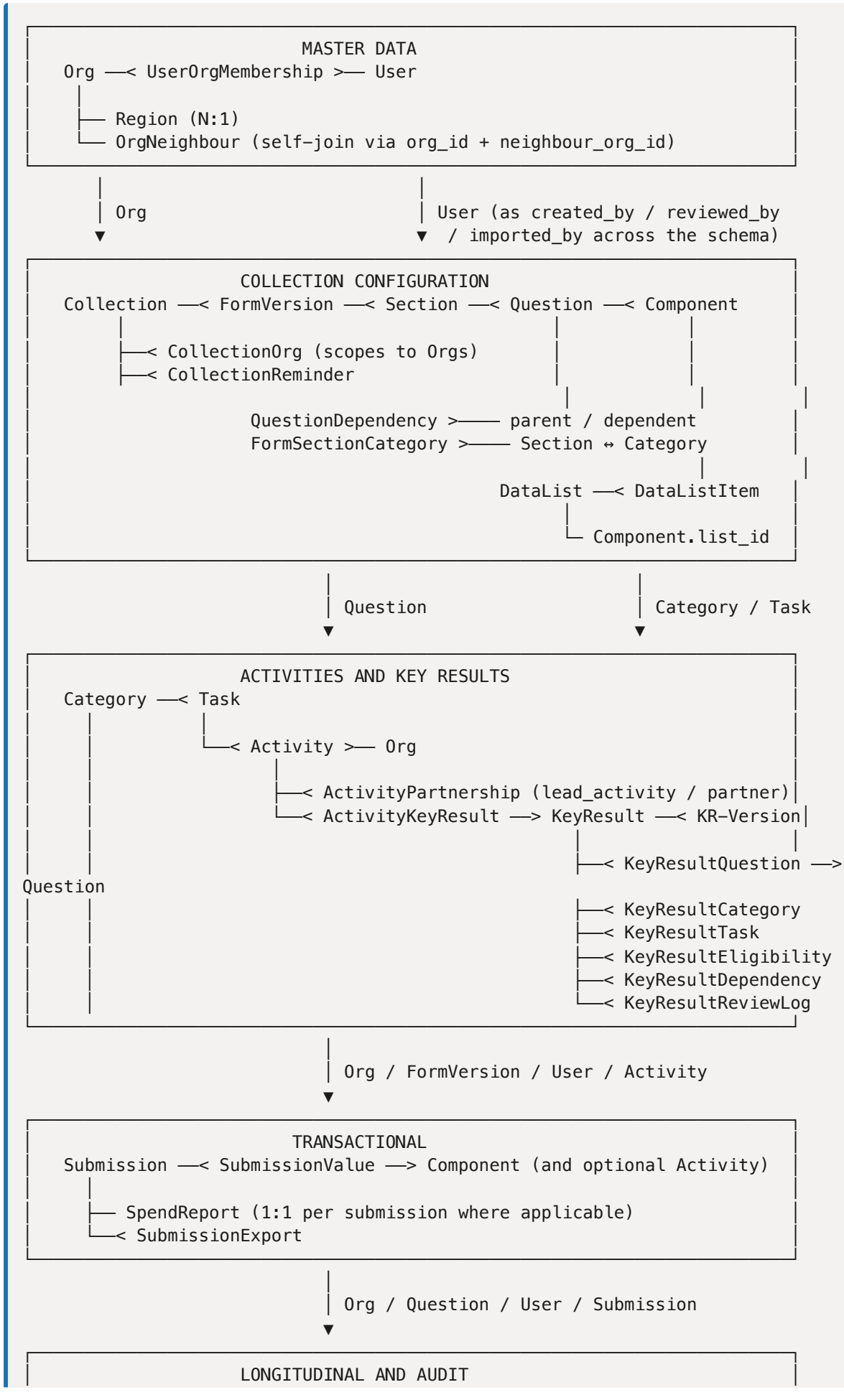
7.5 Longitudinal and audit

Enum	Values
`MaturityStatus`	`draft`, `submitted`, `accepted`
`FeedbackType`	`in_journey`, `post_submission`

---

8. Entity Relationship Overview

A simplified view of domain boundaries and the key cross-domain joins. For the full diagram see Gambling_Levy_Portal_ERD.pdf.


MASTER DATA
Org ──< UserOrgMembership >── User
Region (N:1)
OrgNeighbour (self-join via org_id + neighbour_org_id)
Org
User (as created_by / reviewed_by / imported_by across the schema)
COLLECTION CONFIGURATION
Collection ──< FormVersion ──< Section ──< Question ──< Component
CollectionOrg (scopes to Orgs)
CollectionReminder
QuestionDependency >─── parent / dependent
FormSectionCategory >─── Section ↔ Category
DataList ──< DataListItem
Component.list_id
Question
Category / Task
ACTIVITIES AND KEY RESULTS
Category ──< Task
Activity >── Org
ActivityPartnership (lead_activity / partner)
ActivityKeyResult ──> KeyResult ──< KR-Version
KeyResultQuestion ──>
Question
KeyResultCategory
KeyResultTask
KeyResultEligibility
KeyResultDependency
KeyResultReviewLog
Org / FormVersion / User / Activity
TRANSACTIONAL
Submission ──< SubmissionValue ──> Component (and optional Activity)
SpendReport (1:1 per submission where applicable)
SubmissionExport
Org / Question / User / Submission
LONGITUDINAL AND AUDIT

```
│   MaturityRecord ──< MaturityAnswer ──> Question                          │
│   EventLog (polymorphic: entity_type + entity_id)                        │
│   Feedback ──> Submission / Question / Org / User                        │
└──────────────────────────────────────────────────────────────────────────┘
```

8.1 Canonical ERD

The following pages reproduce the canonical ERD (`Gambling_Levy_Portal_ERD.pdf` , April 2026) for inline reference.

Gambling Levy Portal

Entity Relationship Diagram

Full schema reference — April 2026

Document contents

Page 1 Title page
Page 2 Full ERD overview — all 35 entities and key relationships
Page 3 Master data domain detail
Page 4 Collection configuration domain detail
Page 5 Activities and Key Results domain detail
Page 6 Transactional domain detail
Page 7 Longitudinal and audit domain detail

Domain colour key

Master data

Collection configuration

Activities and Key Results

Transactional

Longitudinal and audit

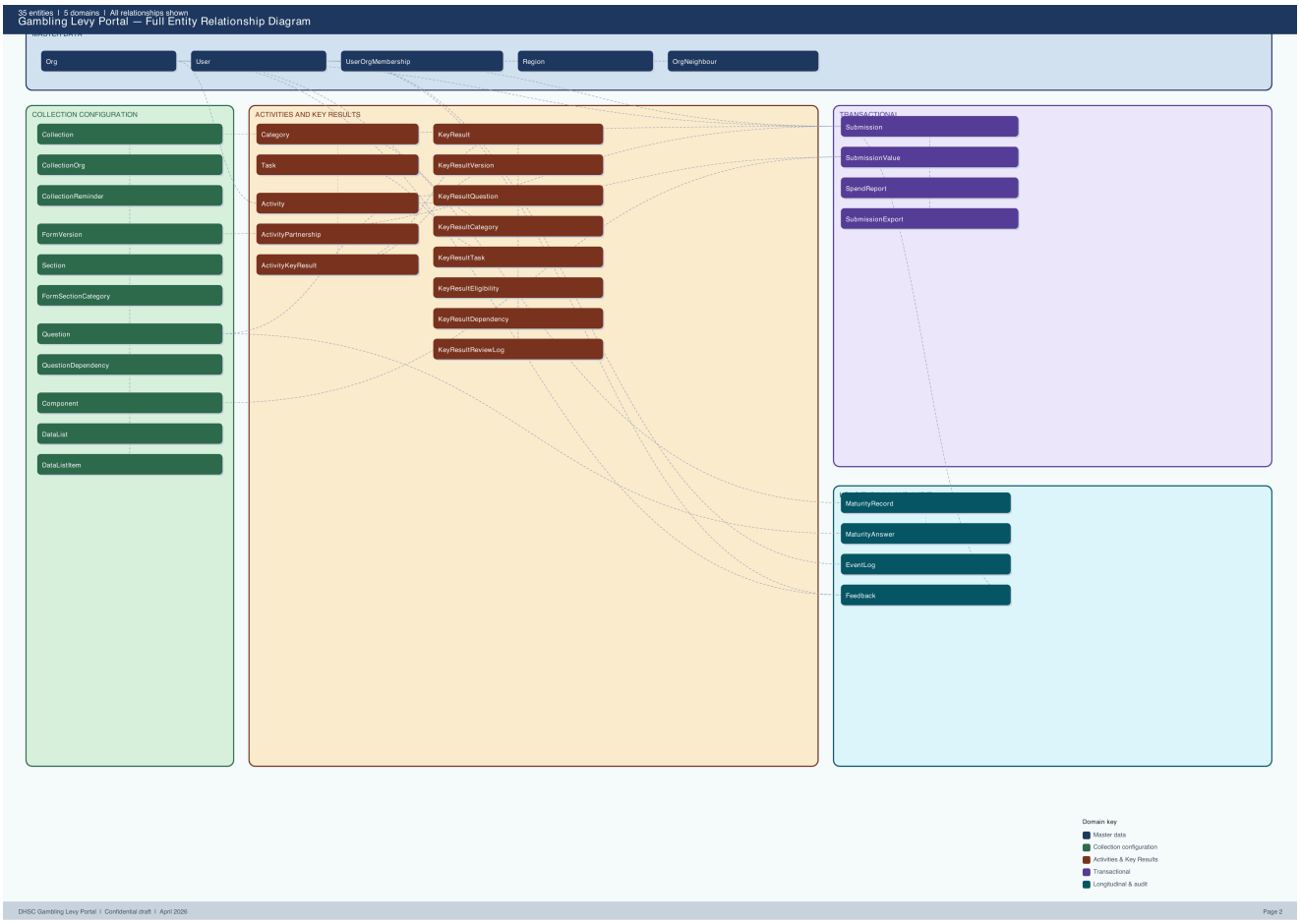
Attribute key

PK Primary key

FK Foreign key

35 entities across 5 domains

5 entities Master data
11 entities Collection configuration
13 entities Activities and Key Results
4 entities Transactional
4 entities Longitudinal and audit


35 entities | 5 domains | All relationships shown
Gambling Levy Portal — Full Entity Relationship Diagram
Org
User
UserOrgMembership
Region
OrgNeighbour
COLLECTION CONFIGURATION
Collection
CollectionOrg
CollectionReminder
FormVersion
Section
FormSectionCategory
Question
QuestionDependency
Component
DataList
DataListItem
ACTIVITIES AND KEY RESULTS
Category
Task
Activity
ActivityPartnership
ActivityKeyResult
KeyResult
KeyResultVersion
KeyResultQuestion
KeyResultCategory
KeyResultTask
KeyResultEligibility
KeyResultDependency
KeyResultReviewLog
TRANSACTIONAL
Submission
SubmissionValue
SpendReport
SubmissionExport
MaturityRecord
MaturityAnswer
EvantLog
Feedback
Domain key
Master data
Collection configuration
Activities & Key Results
Transactional
Longitudinal & audit

Domain: Master data

5 entities

Org

org_id	PK
org_name	
org_type	
region_id	FK
ons_code	
vendor_id	
total_fund	
onboarding_status	
created_at	
updated_at	

User

user_id	PK
full_name	
email	
mfa_status	
contact_number	
job_title	
is_active	
last_login_at	
created_at	

UserOrgMembership

membership_id	PK
user_id	FK
org_id	FK
role	
is_active	
created_at	
updated_at	

Region

region_id	PK
name	
region_type	
created_at	

OrgNeighbour

id	PK
org_id	FK
neighbour_org_id	FK
source	
reference_tool	
imported_at	
imported_by	FK

PK = Primary key FK = Foreign key

Domain: Collection configuration

11 entities

Collection

collection_id	PK
name	
type	
status	
start_date	
end_date	
financial_year	
is_dhsc_managed	
created_by	FK
manually_closed_at	
manually_closed_by	FK
extended_to	
extended_by	FK
created_at	
published_at	

CollectionOrg

collection_org_id	PK
collection_id	FK
org_id	FK
is_required	
due_date_override	
status	
created_at	
updated_at	

CollectionReminder

reminder_id	PK
collection_id	FK
trigger_type	
trigger_value	
message_template	
sent_at	
created_by	FK
created_at	

FormVersion

form_version_id	PK
collection_id	FK
version_number	
version_label	
change_summary	
published_at	
is_active	
created_by	FK
superseded_by	FK
created_at	
updated_at	

Section

section_id	PK
form_version_id	FK
title	
description	
sequence_order	
display_logic	
is_repeatable	
repeat_label	
repeat_min	
repeat_max	
is_optional	
created_by	FK
created_at	

FormSectionCategory

id	PK
section_id	FK
category_id	FK
created_by	FK
created_at	

Question

question_id	PK
section_id	FK
task_id	FK
question_text	
hint_text	
guidance_html	
guard_logic	
display_logic	
is_optional	
is_repeatable	
sequence_order	
tag	
created_by	FK
created_at	
archived_at	

QuestionDependency

dependency_id	PK
parent_question_id	FK
dependent_question_id	FK
condition_operator	
condition_value	
action	
created_by	FK
created_at	

Component

component_id	PK
question_id	FK
type	
list_id	FK
placeholder_text	
prefix_label	
suffix_label	
min_value	
max_value	
min_length	
max_length	
min_date	
max_date	
is_required	
sequence_order	

DataList

list_id	PK
list_name	
description	
is_dhsc_managed	
is_active	
created_by	FK
created_at	
updated_at	

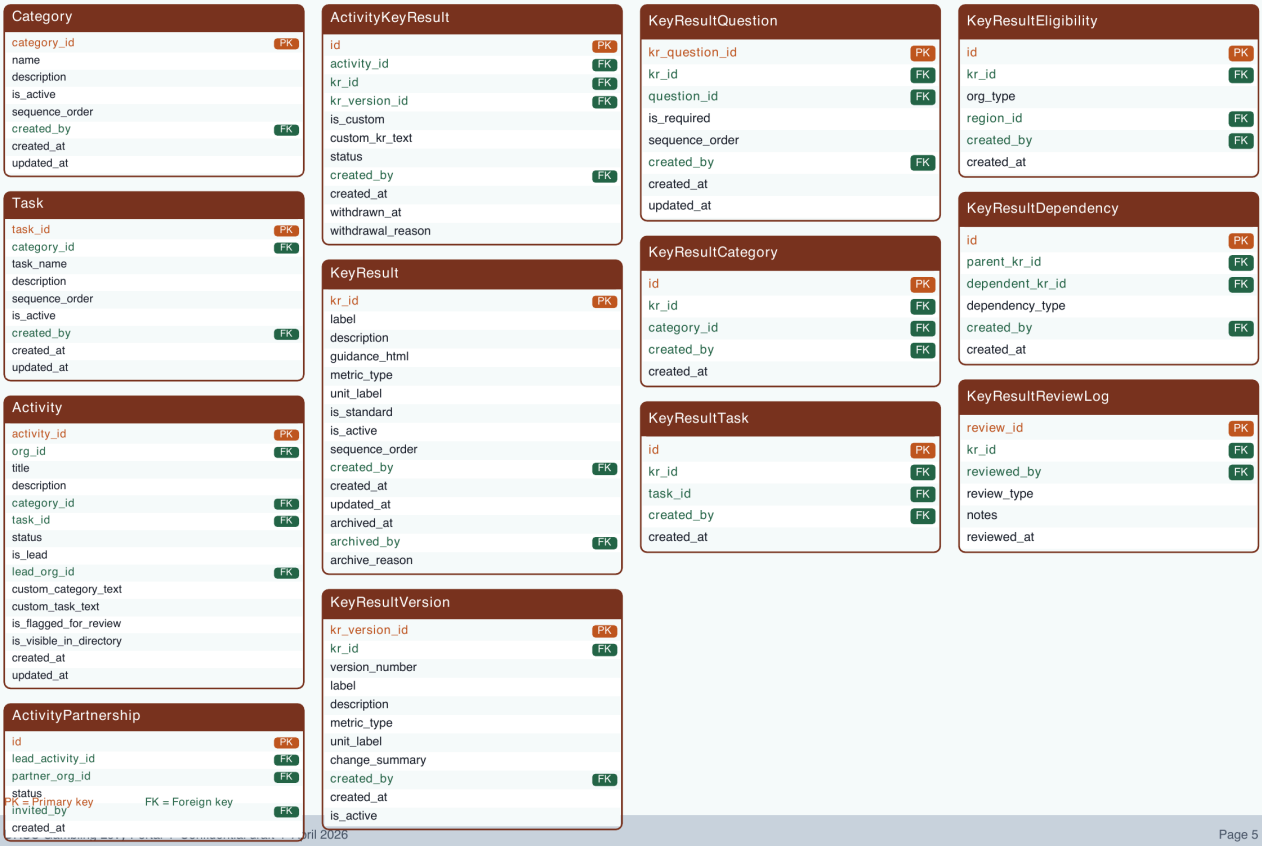
DataListItem

item_id	PK
list_id	FK
label	
value	
hint_text	
sequence_order	
is_active	
created_at	
updated_at	

PK = Primary key FK = Foreign key

Domain: Activities and Key Results

13 entities



Domain: Transactional

4 entities

Submission

submission_id	PK
org_id	FK
collection_id	FK
form_version_id	FK
user_id	FK
proxy_org_id	FK
submitted_at	
status	
snapshot_id	
created_at	
updated_at	

SubmissionValue

value_id	PK
submission_id	FK
component_id	FK
activity_id	FK
value_type	
reported_value	
currency	
created_at	
updated_at	

SpendReport

spend_report_id	PK
submission_id	FK
org_id	FK
reporting_period_start	
reporting_period_end	
total_reported_spend	
submitted_at	
status	

SubmissionExport

export_id	PK
submission_id	FK
org_id	FK
export_type	
generated_by	FK
generated_at	
financial_year	

PK = Primary key FK = Foreign key



9. Examples

9.1 KR-driven quarterly return generation

Demonstrates the central mechanism: Key Results selected against an Activity dynamically generate questions in that Org's quarterly return.

```
Setup (DHSC admin, one-off):
  Category: "Education" (category_id = C1)
  Task: "Workforce training" (task_id = T1, category_id = C1)

  KeyResult (kr_id = KR1, is_standard = true):
    label = "Number of frontline staff trained"
    metric_type = numeric, unit_label = "staff"

  KeyResultVersion (kr_version_id = V1, kr_id = KR1, version_number = 1)

  KeyResultTask: (kr_id = KR1, task_id = T1)

  KeyResultQuestion (kr_id = KR1):
    Q_A: "Number of staff trained this quarter" → Component type = number
    Q_B: "Training delivery format" → Component type = select, list_id = L1
    Q_C: "Narrative: notable successes or challenges" → Component type = textarea, tag
= narrative

Registration (LA, once per activity):
  Activity (activity_id = A1, org_id = ORG1, category_id = C1, task_id = T1)
  ActivityKeyResult (activity_id = A1, kr_id = KR1, kr_version_id = V1)

Quarterly collection opens:
  Collection (type = quarterly, collection_id = Q1-2627)
  FormVersion (form_version_id = FV1, collection_id = Q1-2627)

Form rendering for ORG1:
  The form engine walks ORG1's active ActivityKeyResults.
  For each, it joins KeyResultQuestion on kr_id and renders Q_A, Q_B, Q_C
  inside a repeating Section keyed by activity_id.

Submission:
  Submission (submission_id = S1, org_id = ORG1, collection_id = Q1-2627,
              form_version_id = FV1, status = submitted)

  SubmissionValue × 3 (one per question):
    (submission_id = S1, component_id = Q_A.comp, activity_id = A1, value = "42")
    (submission_id = S1, component_id = Q_B.comp, activity_id = A1, value = "online")
    (submission_id = S1, component_id = Q_C.comp, activity_id = A1,
     value = "Increased reach via regional partners")

  SpendReport (submission_id = S1, total_reported_spend = 18500.00,
               status = submitted)
```

A later DHSC edit to KR1 (e.g. renaming the label) creates `KeyResultVersion V2` but does not mutate V1. ORG1's historic submissions still render against V1 via `ActivityKeyResult.kr_version_id`.

9.2 Partnership activity

```
  Lead (ORG_A):
    Activity (activity_id = A_lead, org_id = ORG_A, is_lead = true, lead_org_id = null)

  Partner (ORG_B):
    ActivityPartnership (lead_activity_id = A_lead, partner_org_id = ORG_B,
                         status = pending, invited_by = USER_A_admin)

  ORG_B accepts:
    ActivityPartnership.status = active

  Form behaviour:
    - ORG_A submits the qualitative narrative for A_lead.
    - ORG_B sees A_lead as read-only in its directory, with Activity.lead_org_id =
ORG_A
      on its synced view.
    - ORG_B still reports its own spend against A_lead in its own Submission.
```

9.3 Proxy submission

```
  User U1 has UserOrgMembership in both ORG_X (role = contributor)
  and ORG_Y (role = lead).

  U1 submits on behalf of ORG_X:
    Submission (org_id = ORG_X,            ← data-owning Org
                user_id = U1,              ← submitter
                proxy_org_id = ORG_Y,      ← Org U1 was acting for
                status = submitted)

  Audit view shows:
    - Data owner: ORG_X
    - Submitted by: U1
    - Acting as: ORG_Y (not self-submission by ORG_X)
```

9.4 Annual Maturity Survey lifecycle

```
1. MaturityRecord (record_id = M1, org_id = ORG1,
                   effective_from = 2026-04-01, effective_to = null,
                   status = draft)

2. LA contributors populate answers throughout the year:
   MaturityAnswer (record_id = M1, question_id = Q_maturity_1, value = "...")
   MaturityAnswer (record_id = M1, question_id = Q_maturity_2, value = "...")
   ...

3. Annual confirmation:
   MaturityRecord.status = submitted
   MaturityRecord.effective_to = 2027-03-31
   A new MaturityRecord M2 is created with effective_from = 2027-04-01
   seeded from M1's answers.

4. Year-on-year comparison queries join MaturityRecord by org_id
   and use effective_from / effective_to to slice by year.
```

---

10. Key Design Decisions

These decisions are canonical — changes to the schema that contradict them should trigger a design-review cycle.

Decision	What we chose	What we rejected	Rationale
Form versioning	`FormVersion` entity with `superseded_by` FK. Each submission records the `form_version_id` it was completed against.	A single mutable form definition per collection.	DHSC needs to update forms between reporting cycles without corrupting historic submissions. Versioning ensures every past submission can be accurately re-rendered.
KR-to-question mapping	`KeyResultQuestion` join table — explicit, admin-editable mapping between a KR and the questions it injects.	Hardcoding KR-to-question logic in application code.	The product requirement is a no-code admin interface. Making this a data relationship means DHSC can add or remove questions from a KR without a code deployment.
KeyResult versioning	`KeyResultVersion` entity. `ActivityKeyResult` stores `kr_version_id` at point of selection.	Mutable `KeyResult` records.	When DHSC updates a KR label or metric type, historic activity registrations must not silently change. Versioning freezes the contract between the LA and DHSC for each activity.
Statistical neighbours	`OrgNeighbour` table with `source` enum. Pairs loaded by DHSC admin from an externally maintained reference dataset.	Deriving statistical neighbours internally from stored attributes (IMD decile, population band, etc.).	A standing government dataset already maintains these groupings. Reimplementing the derivation in-product would duplicate effort and diverge from the agreed methodology over time.
Conditional form logic	`QuestionDependency` table — one row per rule with explicit `condition_operator` and `action`.	Storing all logic as a JSON blob on the `Question` record.	Row-per-rule makes each condition readable, editable, and deletable by admins through a UI without a JSON editor. `Question.guard_logic` is retained as a fallback for complex compound logic.
Partnership activities	`ActivityPartnership` join table with `lead_activity_id`, `partner_org_id`, and `status`.	A single boolean `is_partnership` flag on `Activity`.	The epic requires a structural parent/child relationship — partner LAs receive a synced read-only view of the lead's activity. A boolean flag cannot express the relationship, the partners involved, or the invitation lifecycle.

Decision	What we chose	What we rejected	Rationale
Payment processing	`SpendReport` records what an org spent. Status tracks the digital submission lifecycle only.	`PaymentRun` entity, payment triggers, BACS integration, or frozen payment snapshots.	Payment processing is handled offline by DHSC finance using existing ERP systems. The portal collects and exports spend data; it does not process or trigger payments.
Proxy submissions	`proxy_org_id` nullable FK on `Submission`. `Submission.org_id` always refers to the data-owning Org.	A separate proxy submission table or a user-level flag.	The audit requirement is clear: DHSC must see who submitted the data (User) and which legal entity owns it (Org). Storing `proxy_org_id` on `Submission` captures both in a single record.
DataListItem soft deletion	`is_active` boolean on `DataListItem`. Retired items remain in the database.	Hard deletion of retired list options.	Historic submissions reference specific list values. Hard-deleting a retired option would break the ability to accurately re-render or export past submissions.
CFO sign-off	`SubmissionExport` records generation of the yearly financial summary PDF. The offline signing process is not modelled.	Digital signature integration, signed-document upload, or status tracking of the wet-signature process.	The CFO sign-off requires a wet-ink signature and is returned to DHSC via email. The portal generates the document; it does not manage the offline governance process.
No `FundingAllocation` table	Funding captured as `Org.total_fund` scalar.	A dedicated per-LA per-year allocation table.	v1 does not require multi-year allocation history. If DHSC later needs it, `FundingAllocation` can be added as a child of `Org` without breaking existing queries.
Maturity stored separately from `Submission`	Dedicated `MaturityRecord` + `MaturityAnswer` with `effective_from` / `effective_to`.	Modelling maturity as just another `Submission` / `SubmissionValue`.	The maturity survey is a living, year-round profile with annual confirmation — a different lifecycle to quarterly submissions. Co-mingling them would complicate both querying and display.
Polymorphic `EventLog`	Single audit table with `entity_type` + `entity_id`.	Per-entity audit tables.	Regulatory traceability requires uniform coverage. A polymorphic table makes cross-entity audit reports

Decision	What we chose	What we rejected	Rationale
			trivial and avoids per-entity drift.

---

11. Open Questions and Assumptions

The following items were noted during ERD development and require confirmation from DHSC before build commences.

1. **Statistical neighbour data refresh.** Source dataset, file format, and refresh frequency for `OrgNeighbour` imports are not yet confirmed. v1 assumes a manual admin upload.
2. **Partnership activities scope for MVP.** `ActivityPartnership` is included in the ERD for completeness, but the partnership epic may be deferred post-MVP. Confirm whether v1 build includes it.
3. **Custom KR review workflow.** The schema supports the "Other" workflow via `ActivityKeyResult.is_custom` / `custom_kr_text` and `Activity.is_flagged_for_review`, but the admin-facing triage and approval workflow is not yet fully specified.
4. **VCSE maturity tracking.** Maturity is currently scoped to Local Authorities only — VCSEs are excluded from `MaturityRecord`. Confirm whether this is a deliberate long-term decision or a temporary scope limitation.
5. **Feedback data destination.** Whether `Feedback` is the primary store or whether data is also pushed to an external tool (MS Forms, PowerBI, etc.) for weekly DHSC review is not yet confirmed.
6. **`Org.total_fund` vs multi-year history.** The v1 scalar does not capture per-year allocation history. Confirm whether historic allocations are needed in reporting outputs before build.
7. **`MfaStatus` enum values.** Final enum values should be reconciled against MFA-Implementation.md once the MFA domain model is merged into the main schema.
8. **`FormDefinition` grouping.** Prior drafts grouped `FormVersion`s under a `FormDefinition` template entity. The April 2026 ERD ties `FormVersion` directly to `Collection`. Confirm no cross-collection form reuse is required.

Glossary

Term	Definition
DHSC	Department of Health and Social Care — the commissioning body for the portal.
OHID	Office for Health Improvements and Disparities — the DHSC group responsible for the gambling levy programme.
LA	Local Authority. One of ~153 councils required to report on gambling harms prevention work.
VCSE	Voluntary, Community and Social Enterprise — secondary user group alongside Local Authorities.
KR / Key Result	A standardised measurable outcome defined by DHSC that an Org commits to tracking when registering an Activity.
Maturity survey	The annual strategic profile of an LA's approach to gambling harms — updated throughout the year and confirmed annually.
Quarterly return	The recurring three-month financial and performance report submitted by LAs and VCSEs against their active Activities.
Proxy submission	When a User submits data on behalf of an Org other than their own primary employer.
Statement of Truth	The annual financial assurance document generated by the portal (as a `SubmissionExport` of type `yearly_summary_pdf`) and signed offline by the LA's Chief Financial Officer.
Guard logic	Conditional rules that show, hide, require, or disable questions based on previous answers.
Soft deletion	Marking a record as inactive (`is_active = false` or `archived_at = <timestamp>`) rather than removing it from the database, preserving referential integrity for historic data.
Vendor ID	DHSC's internal financial identifier for each Org, used in spend exports to finance systems.
ONS code	Office for National Statistics geographic identifier. Used to map Orgs to regions and statistical classifications.

Changelog

Date	Version	Changes
2026-06-10	2.1	Removed `OrgNeighbour.reference_tool` and all named references to the Children's Services Benchmarking Tool. Statistical-neighbour data is now described generically as an admin-loaded reference dataset (§2.5, §10, §11). Note: the ERD PDF and Excel table still carry the `reference_tool` column — they need to be regenerated from the source modelling tool to match.
2026-04-21	2.0	Full realignment to the April 2026 ERD. Replaced the prior dynamic-forms + financial-returns model with the 35-entity, 5-domain model spanning Master data, Collection configuration, Activities & Key Results, Transactional, and Longitudinal & audit. Added Activities / Key Results domain (previously absent), Annual Maturity Survey entities, polymorphic `EventLog`, `Feedback`, partnership model, proxy submissions, and offline CFO sign-off. Removed `FormDefinition`, `FundingAllocation`, `FinancialReturn`, `ExpenditureLine`, `AnnualReconciliation` — all superseded. Split the former `FormField` into `Question` + `Component`. Renamed `DataSource` → `DataList`, `Initiative` → `Activity`, `Tenant` → `Org`, `CollectionWindow` → `Collection`.
2026-02-20	1.0	Initial schema documentation (superseded).